# El Desarrollo de la Ingeniería de Software: Historia, Técnicas y su Impacto en la Ciencia de la Computación

Javier Alejandro González Díaz

*Grupo: C-412*

20 de abril de 2026

**Resumen**

Aquí va el resumen del trabajo. Debe ser conciso y presentar los aspectos más relevantes del tema desarrollado, incluyendo los objetivos, metodología y principales conclusiones. El resumen no debe exceder las 250 palabras y debe proporcionar una visión general clara del contenido del informe.

**Palabras clave:** Ingeniería de Software, Desarrollo, Metodologías, Historia, Técnicas

# Índice

**1. Introducción** **3**

**2. Orígenes y evolución histórica de la ingeniería de software** **3**

2.1. Décadas de 1960 y 1970: primeros modelos de ciclo de vida . . . . . 3

2.2. Décadas de 1980 y 1990: profesionalización y calidad . . . . . 4

2.3. Décadas de 2000 y 2010: agilidad y automatización . . . . . 4

2.4. Síntesis histórica . . . . . 5

**3. Principios fundamentales de la ingeniería de software** **5**

**4. Desarrollo Técnico** **7**

4.1. Fundamentos Teóricos . . . . . 7

4.2. Técnicas y Metodologías . . . . . 7

4.3. Aplicaciones Prácticas . . . . . 7

**5. Impacto en la Ciencia de la Computación** **7**

**6. Conclusiones** **8**

# 1. Introducción

En esta sección se presenta una introducción al tema, estableciendo el contexto histórico y la relevancia del mismo en el campo de la Ciencia de la Computación y la Ingeniería de Software. Se pueden mencionar las motivaciones para el desarrollo del tema y su impacto en la disciplina.

Diversos autores han contribuido al desarrollo de esta área.

# 2. Orígenes y evolución histórica de la ingeniería de software

La ingeniería de software surgió como respuesta a la necesidad de transformar la programación, que durante sus primeras etapas fue una actividad predominantemente artesanal, en una disciplina capaz de producir sistemas complejos con mayor control, calidad y previsibilidad. En los inicios de la computación, especialmente en las décadas de 1940 y 1950, el desarrollo de programas se realizaba con poca estandarización, escasa documentación y casi sin métodos formales de diseño, lo que hacía muy difícil el mantenimiento y la evolución de los sistemas [1].

Con el crecimiento del tamaño y la complejidad de los sistemas informáticos, comenzaron a aparecer numerosos problemas de retrasos, sobrecostos, fallos de fiabilidad y dificultad para adaptar el software a nuevas necesidades. Esta situación dio lugar a lo que posteriormente se denominó la crisis del software, un fenómeno ampliamente reconocido desde finales de los años 60 y consolidado en la Conferencia de la NATO sobre Ingeniería de Software de 1968, donde se propuso tratar el desarrollo de software como una verdadera disciplina de ingeniería [2]. A partir de ese momento, el término "ingeniería de software" comenzó a difundirse para designar un enfoque sistemático y disciplinado de construcción de programas [3].

## 2.1. Décadas de 1960 y 1970: primeros modelos de ciclo de vida

Durante las décadas de 1960 y 1970, el aumento de la complejidad de los sistemas impulsó la búsqueda de modelos de desarrollo más estructurados. En este contexto apareció el modelo en cascada, una propuesta secuencial que divide el proceso en fases bien definidas, como análisis de requisitos, diseño, implementación, pruebas y mantenimiento [4]. Aunque más tarde fue criticado por su rigidez, el modelo en cascada fue importante porque introdujo una visión ordenada del ciclo de vida del software y reforzó la idea de que cada etapa debía documentarse y validarse antes de avanzar a la siguiente [5].

En estos años también se fortaleció el énfasis en la documentación, las pruebas y la modularidad. La documentación se consideró esencial para facilitar la comunicación

entre desarrolladores, usuarios y gestores, así como para asegurar la mantenibilidad de los sistemas [1]. Las pruebas comenzaron a consolidarse como una actividad indispensable para detectar errores antes de la entrega, y la modularidad se presentó como una estrategia para dividir sistemas grandes en componentes más manejables y reutilizables [6].

## 2.2. Décadas de 1980 y 1990: profesionalización y calidad

A partir de los años 80, la ingeniería de software entró en una etapa de mayor profesionalización. El crecimiento de la industria del software exigió prácticas más maduras de planificación, gestión, control de calidad y mejora continua. Uno de los marcos más influyentes fue el Capability Maturity Model (CMM), desarrollado en el Software Engineering Institute, que propuso niveles de madurez para evaluar y perfeccionar los procesos de desarrollo [7]. Este modelo influyó profundamente en la manera en que las organizaciones comenzaron a ver la calidad no solo como una propiedad del producto final, sino también del proceso que lo produce [8].

En paralelo, se consolidaron estándares internacionales como los de la familia ISO/IEC, orientados a establecer criterios comunes para la calidad del software y de los procesos asociados [9]. Estos estándares ayudaron a formalizar prácticas de desarrollo, prueba, mantenimiento y gestión de la configuración, favoreciendo una visión más industrial del software [1]. Durante este período también se expandió la Programación Orientada a Objetos (POO), cuyos principios de encapsulación, herencia y polimorfismo facilitaron la construcción de sistemas más flexibles y reutilizables [10]. Asimismo, aparecieron las herramientas CASE (Computer-Aided Software Engineering), diseñadas para apoyar de forma automatizada tareas de análisis, diseño, documentación y mantenimiento [5].

## 2.3. Décadas de 2000 y 2010: agilidad y automatización

En los años 2000 y 2010, la ingeniería de software experimentó una transformación importante debido a la necesidad de adaptarse a entornos de cambio rápido. Frente a los modelos pesados y rígidos, cobraron protagonismo las metodologías ágiles, que proponen un desarrollo iterativo, incremental y centrado en la colaboración continua con el cliente [11]. Entre los marcos más influyentes destacan Scrum y Extreme Programming (XP), ambos orientados a obtener retroalimentación frecuente, responder con rapidez a cambios en los requisitos y entregar valor en ciclos cortos [12, 13].

En esta misma etapa surgieron prácticas que transformaron la producción y operación del software, especialmente DevOps, que promueve la integración entre desarrollo y operaciones para acelerar la entrega y mejorar la calidad mediante automatización y monitoreo continuo [14]. La integración continua permitió validar de forma frecuente los cambios en el código mediante pruebas automáticas, reduciendo errores e incrementando la confiabilidad [15]. Posteriormente, la infraestructura como código amplió esta lógica

al terreno de la administración de sistemas, permitiendo definir y reproducir entornos de despliegue mediante archivos versionados y automatizados [16].

### 2.4. Síntesis histórica

En síntesis, la evolución de la ingeniería de software refleja el paso desde una práctica artesanal hacia una disciplina madura, apoyada en métodos, estándares, herramientas y procesos cada vez más sofisticados. Cada etapa histórica respondió a desafíos concretos de su tiempo: primero, la necesidad de organizar la programación; después, la de garantizar calidad y mantenimiento; más tarde, la de mejorar la adaptabilidad; y finalmente, la de automatizar la entrega y operación de sistemas complejos [1, 5]. Por ello, la ingeniería de software constituye hoy una de las bases fundamentales de la computación moderna [3].

Figura 1: Línea de tiempo de la evolución de la Ingeniería de Software

**Versión interactiva:**
La versión interactiva de la línea de tiempo se encuentra disponible en línea, permitiendo una exploración dinámica de las distintas etapas históricas de la ingeniería de software. Abrir línea de tiempo interactiva

## 3. Principios fundamentales de la ingeniería de software

La ingeniería de software se concibe como una disciplina **sistemática, disciplinada y cuantificable** porque su objetivo no es solo programar, sino construir productos de software con calidad predecible, costo controlado y capacidad de evolución [1]. Esta visión implica que el desarrollo debe apoyarse en métodos definidos, procesos repetibles y métricas que permitan evaluar tanto el producto como el proceso de construcción [5].

Un principio central es que el software debe satisfacer tanto los **requisitos funcionales** como los **no funcionales** [1]. Los requisitos funcionales describen qué debe hacer el sistema, por ejemplo, procesar datos, autenticar usuarios o generar reportes; los no funcionales especifican cómo debe comportarse, incluyendo rendimiento, seguridad, disponibilidad, portabilidad y facilidad de uso [5]. La ingeniería de software busca equilibrar ambos tipos de requisitos, porque un sistema puede cumplir su función principal y aun así fracasar si no responde adecuadamente a restricciones de calidad o de contexto [1].

La **calidad del software** es otro principio esencial. No se limita a la ausencia de errores, sino que abarca atributos como **mantenibilidad, fiabilidad, usabilidad, seguridad y eficiencia** [9]. La mantenibilidad permite corregir y evolucionar el sistema con menor esfuerzo; la fiabilidad garantiza que el software funcione de manera consistente; la usabilidad facilita que los usuarios interactúen con él de forma eficaz; la seguridad protege la información y los recursos; y la eficiencia busca un uso adecuado del tiempo de procesamiento, memoria y otros recursos [5, 9]. En la práctica, estos atributos se evalúan mediante revisiones, pruebas, métricas y estándares de calidad [1].

El **ciclo de vida del software** organiza el trabajo en etapas sucesivas y relacionadas entre sí. De forma general, este ciclo incluye **requerimientos, diseño, implementación, pruebas, despliegue y mantenimiento** [5]. En la fase de requerimientos se identifican las necesidades del usuario y las restricciones del sistema; en el diseño se define la estructura de la solución; en la implementación se traduce el diseño al código; en las pruebas se verifica el cumplimiento de los requisitos; en el despliegue se entrega el sistema al entorno real; y en el mantenimiento se corrigen errores y se incorporan mejoras [1]. Esta secuencia no debe entenderse solo como una lista de pasos, sino como una forma de controlar la complejidad y asegurar trazabilidad entre lo que se necesita y lo que finalmente se construye [5].

Entre los principios de diseño más importantes se encuentra la **modularidad**, que consiste en dividir el sistema en partes relativamente independientes [6]. La modularidad facilita la comprensión, prueba y mantenimiento del software, porque permite trabajar sobre componentes más pequeños y delimitados [6]. Relacionado con ello está la **cohesión**, que expresa el grado en que los elementos de un módulo están enfocados a una sola responsabilidad; una cohesión alta suele favorecer la claridad del diseño [5]. En contraste, el **acoplamiento bajo** busca minimizar las dependencias entre módulos, de manera que un cambio en una parte del sistema afecte lo menos posible a las demás [1].

Otro principio fundamental es la **reutilización**, es decir, la posibilidad de emplear componentes, clases, bibliotecas o servicios en diferentes contextos [10]. La reutilización reduce costos, acelera el desarrollo y puede mejorar la confiabilidad cuando los componentes reutilizados ya han sido probados en otros proyectos [17]. Finalmente, la **extensibilidad** describe la capacidad del software para incorporar nuevas funciones sin necesidad de reescribir gran parte del sistema [1]. Este principio es especialmente valioso en entornos cambiantes, donde los requisitos evolucionan con rapidez y el software debe adaptarse sin perder estabilidad [5].

En conjunto, estos principios muestran que la ingeniería de software no consiste únicamente en producir código, sino en construir soluciones correctas, útiles, mantenibles y adaptables. Por ello, la calidad del resultado depende tanto de la gestión del proceso como de las decisiones de diseño adoptadas desde las primeras etapas del desarrollo [1, 5].

# 4. Desarrollo Técnico

Esta sección aborda los aspectos técnicos del tema, incluyendo:

## 4.1. Fundamentos Teóricos

Descripción de los fundamentos teóricos y conceptos clave. Se pueden incluir fórmulas matemáticas cuando sea necesario.

$$f(x) = \sum_{i=1}^{n} a_i x^i \tag{1}$$

La ecuación 1 representa...

## 4.2. Técnicas y Metodologías

Descripción detallada de las técnicas, metodologías o algoritmos relacionados con el tema.

```
// Ejemplo de pseudocódigo
function ejemplo(parametro):
    resultado = procesar(parametro)
    return resultado
```

## 4.3. Aplicaciones Prácticas

Ejemplos de aplicación práctica de las técnicas descritas en contextos reales de desarrollo de software.

Cuadro 1: Ejemplo de tabla comparativa

| **Característica** | **Opción A** | **Opción B** |
|---|---|---|
| Rendimiento | Alto | Medio |
| Complejidad | Baja | Alta |
| Escalabilidad | Buena | Excelente |

Como se muestra en la Tabla 1, ...

# 5. Impacto en la Ciencia de la Computación

Análisis del impacto y relevancia del tema en el desarrollo de la Ciencia de la Computación y la Ingeniería de Software. Se debe discutir:

1. Contribuciones principales al campo

2. Influencia en tecnologías actuales
3. Perspectivas futuras
4. Relación con otras áreas de conocimiento

El trabajo de [] demuestra la importancia de...

# 6. Conclusiones

Síntesis de los aspectos más importantes desarrollados en el trabajo, destacando las contribuciones del tema a la Ingeniería de Software y las lecciones aprendidas.

# Referencias


[1] I. Sommerville, *Software Engineering*. Harlow, England: Pearson, 10 ed., 2016. Página oficial de la editorial Pearson.

[2] P. Naur and B. Randell, "Software engineering: Report on a conference sponsored by the nato science committee," in *NATO Science Committee Conference on Software Engineering*, (Garmisch, Germany), NATO, 1969. Versión digital disponible en línea.

[3] B. W. Boehm, *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice Hall, 1981.

[4] W. W. Royce, "Managing the development of large software systems," *Proceedings of IEEE WESCON*, pp. 1–9, 1970. Copia en línea del artículo.

[5] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*. New York, NY: McGraw-Hill Education, 9 ed., 2020. Página oficial de la editorial McGraw-Hill.

[6] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[7] M. C. Paulk, B. Curtis, M. B. Chrissis, and C. V. Weber, *Capability Maturity Model for Software, Version 1.1*. Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 1993. Documento del SEI.

[8] W. S. Humphrey, *Managing the Software Process*. Reading, MA: Addison-Wesley, 1989.

[9] "Iso/iec 12207: Information technology – software life cycle processes," 1995. Página oficial de ISO.

[10] G. Booch, *Object-Oriented Analysis and Design with Applications*. Redwood City, CA: Addison-Wesley, 2 ed., 1994.

[11] Agile Alliance, "Manifesto for agile software development." https://agilemanifesto.org, 2001. Accedido: 2026-04-20.

[12] K. Schwaber and J. Sutherland, *The Scrum Guide*. Scrum.org, 2020. Accedido: 2026-04-20.

[13] K. Beck, *Extreme Programming Explained: Embrace Change*. Boston, MA: Addison-Wesley, 2 ed., 2004.

[14] G. Kim, P. Debois, J. Willis, and J. Humble, *The DevOps Handbook*. Portland, OR: IT Revolution Press, 2016. Página oficial del editor.

[15] M. Fowler, "Continuous integration." https://martinfowler.com/articles/continuousIntegration.html, 2006. Accedido: 2026-04-20.

[16] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA: Addison-Wesley, 2010. Página de acceso de O'Reilly.

[17] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley Professional, 1994.